

ALGORITHM NOTES

卡码学院 笔记合集

个人学习记录

AUTHOR

axucoder

GENERATED

2026-07-01

PLATFORM

卡码学院

```
// keep solving, keep growing
function solve(problem) {
  while (!solved) {
    think(); code(); debug();
  }

  return "AC";
}
```

代码随想录

数组

《代码随想录》数组：数组理论基础

在C++中二维数组在地址空间上是连续的，且数组元素不能删除只能覆盖

《代码随想录》数组：二分查找

二分法

大家写二分法经常写乱，主要是因为**对区间的定义没有想清楚，区间的定义就是不变量**。要在二分查找的过程中，保持不变量，就是在while寻找中每一次边界的处理都要坚持根据区间的定义来操作，这就是**循环不变量**规则。写二分法，区间的定义一般为两种，左闭右闭即[left, right]，或者左闭右开即[left, right)。

左闭右闭写法

while (left <= right) 要使用 <=，因为left == right是有意义的，所以使用 <= if (nums[middle] > target) right 要赋值为 middle - 1，因为当前这个nums[middle]一定不是target，那么接下来要查找的左区间结束下标位置就是 middle - 1

下面是左闭右闭的C++代码实现

```
// 多行代码块
class Solution {
    public int search(vector<int>& nums, int target) {
        int left = 0, right = nums.length - 1;
        while(left ≤ right){
            // 获取中间值
            int mid = (right + left) / 2 ;
            // 比较nums[mid]和target的值, 如果大于, 说明要在左侧寻找
            if(nums[mid] > target){
                right = mid - 1; // 右侧指针指向mid左侧
            }else if(nums[mid] < target){
                left = mid + 1; // 同理:
            }else{
                return mid;
            }
        }
        // 正常无法抵达这个位置:
        return -1;
    }
}
```

左闭右开写法

```
// 多行代码块
class Solution {
    public int search(vector<int>& nums, int target) {
        int left = 0, right = nums.size() - 1;
        while(left < right){
            // 获取中间值
            int mid = (right + left) / 2 ;
            // 比较nums[mid]和target的值, 如果大于, 说明要在左侧寻找
            if(nums[mid] > target){
                right = mid ; // 右侧指针指向mid左侧
            }else if(nums[mid] < target){
                left = mid + 1 ; // 同理:
            }else{
                return mid;
            }
        }
        // 正常无法抵达这个位置:
        return -1;
    }
}
```

while (left < right), 这里使用 <, 因为 left == right 在区间 [left, right) 是没有意义的 if (nums[middle] > target) right 更新为 middle, 因为当前 nums[middle] 不等于 target, 去左区间继续寻找, 而寻找区间是左闭右开区间, 所以 right 更新为 middle, 即: 下一个查询区间不会去比较 nums[middle]

《代码随想录》数组：移除元素

常规方法：暴力法

使用两层 for 循环进行求解, 较好想到。外层循环用来遍历整个数组, 当遍历到 target 时, 使用内层循环将其后面的元素全部依次前移一位即可。

```
for (int i = 0; i < size; i++) {
    if (nums[i] == val) {
        for (int j = i + 1; j < size; j++) {
            nums[i] = nums[j];
        }
        i--;
        size--;
    }
}
```

```### 新方法：双指针法

考虑到两层 for 循环带来的  $O(n^2)$  的时间复杂度, 想一想是否能将其简化为  $O(n)$  的呢?

因此考虑快慢指针法, 其中 fast 指针的作用类似于常规方法中外层循环的作用, 主要用来遍历数组, 取出数组中值不等于 val 的元素, 而 slow 指针的作用则是将不等于 val 的元素移动到 slow 的位置。简单来说, 我们可以假设有两个数组, 第一个就是旧数组, 包含全部元素; 第二个则是新数组, 仅包含删除后的元素。我们

for (int fast = 0; fast < size; fast++) { if (nums[fast] != val) { nums[slow] = nums[fast]; slow++; } } 看这里的代码, 可以看到当 fast 指向一般元素时, 我们让 nums[slow] = nums[fast], 即覆盖了一次 (开始的时候是原地覆盖), 其实也就是放入了一个新元素到数组中, 只不过这个数组就是原数组罢了, 当 fast 指向 target 时, 不做任何处理, 直接跳过, slow 也就指向 target 了, 之后当 fast 再指向一般元素时, 就将 slow 处的元素覆盖了, 也就是移除了目标值元素。实际上, 不需要将 target 看成什么很特殊的元素, 我们就将其看作空位, 当遇到空位时, 直接跳过就可以了, 当遇到的不是空位时, 就让它占上前面空的位置。

## 《代码随想录》数组：有序数组的平方

""" 题干: 给你一个按非递减顺序排序的整数数组 nums, 返回每个数字的平方组成的新数组, 要求也按非递减顺序排序。 """

```

class Solution {
public:
 vector<int> sortedSquares(vector<int>& A) {
 int k = A.size() - 1;
 vector<int> result(A.size(), 0);
 for (int i = 0, j = A.size() - 1; i <= j;) { // 注意这里要i <= j, 因为最后要处理两个元素
 if (A[i] * A[i] < A[j] * A[j]) {
 result[k--] = A[j] * A[j];
 j--;
 }
 else {
 result[k--] = A[i] * A[i];
 i++;
 }
 }
 return result;
 }
};

```

暴力解 这道题第一思路是遍历数组平方后，再使用sorted函数排序；时间复杂度是 $O(n \log n)$

```

class Solution {
public:
 vector<int> sortedSquares(vector<int>& A) {
 for (int i = 0; i < A.size(); i++) {
 A[i] *= A[i];
 }
 sort(A.begin(), A.end()); // 快速排序
 return A;
 }
};

```

时间复杂度为 $O(n)$ 的思路是双指针法。原数组有序，所以最大平方值一定出现在数组两端（左端有较大负数或右端有较大正数）。使用左右指针分别指向数组开头和结尾，比较左右指针指向元素的平方值。将较大的平方值放入结果数组的末尾，移动指针。

## 《代码随想录》数组：长度最小的子数组

### 1.暴力解

两个for循环枚举出所有情况

### 2.为什么想到了滑动窗口：

碰到这种需要遍历且要求一个数组形式的各种数值 就可以考虑到滑动窗口( 可以理解为 当快的指针遍历到题目所求时 需要让慢指针来不断向前移动以缩小/靠近题目的范围最终得到结果 )

## 2.代码详细思路

本题求的是数组元素总和  $\geq$  目标值 返回对应的数组长度 因此一定要维护两个变量 sum 和 result 根据 1. 中所提 自然想到 fast 和 slow 两个指针 fast 指针用于遍历整个数组不断更新 sum 的值 当  $\text{sum} \geq \text{target}$  时 题目要求得到最小长度 那么就需要 slow 不断的向前移动获得最小值 减少 sum 的数值 直到 sum 再次小于 target 按照这样的循环流程 遍历完整整个数组 最小长度自然在 result 中 那么需要考虑以下几个因素

1. 初始化 sum 和 result sum 方面没什么好说的 直接  $\text{sum} = 0$  result 方面: 看题目要求的最后一个 需要考虑到整个数组的元素总和不足 target 的情况 且要考虑到 result 更新时要默认两者取最小, 那么 result 就不能干扰结果的获得 result 需要标记为最大值 那么如何获得返回值: 只要看 result 是否一直在更新即可, 因为需要满足  $\text{sum} > \text{target}$  的状况下 result 才会更新 没更新自然说明不满足以上条件 直接指定  $\text{result} = \text{MAX\_VALUE} ? 0 : \text{result}$  即可
2. 何时更新 slow 和 fast fast 也没什么好说的 满足  $\text{sum} < \text{target}$  后不断更新即可(实际代码写的时候甚至可以不说条件) slow 满足  $\text{sum} \geq \text{target}$  后不断更新即可:

**易错点** slow 需要循环 因为我们无法保证只向右移动一次就算满足条件 不是 if 而是 while

成品代码展示:

```
class Solution {
 public int minSubArrayLen(int target, int[] nums) {
 int slow = 0, fast = 0, n = nums.length;
 int result = Integer.MAX_VALUE, sum = 0;
 for(; fast < n; fast++){
 sum += nums[fast];
 while(sum >= target){
 result = Math.min(result, fast - slow + 1); // 先更新数值:
 sum -= nums[slow]; // 减去对应数值;
 slow++;
 }
 }
 return result == Integer.MAX_VALUE ? 0 : result;
 }
}
` `` ##### 时间复杂度 -- $O(N)$
```

不要以为 for 里放一个 while 就以为是  $O(n^2)$  啊, 主要是看每一个元素被操作的次数, 每个元素在滑动窗后进来操作一次

### [《代码随想录》数组: 螺旋矩阵II](<https://notes.kamacoder.com/questions/501951>)

# 一、题目描述

给定一个正整数 `n`, 生成一个 `n × n` 的矩阵, 元素为 `1 ~ n^2`, 并按照 \*\*顺时针螺旋顺序\*\* 填充。  
例如:

n = 3

输出

[ [1,2,3], [8,9,4], [7,6,5] ]

----

# 二、核心思路（按圈模拟）

矩阵可以看成 **\*\*一圈一圈填充\*\***。

每一圈分为 **\*\*四条边\*\***：

**1** 上边：左 → 右

**2** 右边：上 → 下

**3** 下边：右 → 左

**4** 左边：下 → 上

填完一圈后：

start++ offset++

进入下一圈。

循环次数：

loop = n / 2

因为每次填一圈。

----

# 三、变量含义

|变量|含义|  
|:-:|:-:|  
|res|结果矩阵|  
|startx,starty|每一圈起点|  
|loop|循环圈数|  
|offset|控制右边界|  
|count|当前填入数字|  
|row,col|遍历变量|

----

# 四、执行流程示意 (n=5)

第一圈

1 2 3 4 5 16 6 15 7 14 8 13 12 11 10 9

第二圈

17 18 19 24 20 23 22 21

第三圈

25

----

# 五、注意事项（非常重要）

### 1 循环次数

loop = n / 2

因为每次填一圈。

----

### 2 每条边 \*\*左闭右开\*\*

例如：

for(col=starty; col < n-offset; col++)

这样可以 \*\*避免重复覆盖角落元素\*\*。

----

### 3 更新边界

每填一圈：

startx++ starty++ offset++

----

### 4 奇数矩阵要单独处理中心

例如:

$n = 3$

中心元素:

`res[n/2][n/2]`

代码:

`if(n % 2 == 1) res[n/2][n/2] = count;`

----

### 5 第二条边容易写错

很多人写成

`res[row][col]`

但 `col` 已经固定了。

所以必须注意 **\*\*循环结束后的 col 位置\*\***。

----

# 六、完整代码实现（推荐版本）

```
class Solution { public: vector<vector> generateMatrix(int n) {
```

```

vector<vector<int>> res(n, vector<int>(n, 0));

int startx = 0, starty = 0;
int loop = n / 2;
int count = 1;
int offset = 1;
int row, col;

while (loop--) {

 // 上边
 for (col = starty; col < n - offset; col++) {
 res[startx][col] = count++;
 }

 // 右边
 for (row = startx; row < n - offset; row++) {
 res[row][col] = count++;
 }

 // 下边
 for (; col > starty; col--) {
 res[row][col] = count++;
 }

 // 左边
 for (; row > startx; row--) {
 res[row][col] = count++;
 }

 startx++;
 starty++;
 offset++;
}

// 奇数中心
if (n % 2 == 1) {
 res[n/2][n/2] = count;
}

return res;
}

```

```

};

```

时间复杂度：

$O(n^2)$

空间复杂度：

$O(n^2)$

----

### # 七、另一种更容易写的写法（推荐）

面试中很多人更喜欢 **四边界写法**。

思路：

维护四个边界：

top bottom left right

代码：

```
class Solution { public: vector<vector> generateMatrix(int n) {
```

```

vector<vector<int>> res(n, vector<int>(n));

int left = 0, right = n - 1;
int top = 0, bottom = n - 1;

int num = 1;

while (left ≤ right && top ≤ bottom) {

 for (int j = left; j ≤ right; j++)
 res[top][j] = num++;
 top++;

 for (int i = top; i ≤ bottom; i++)
 res[i][right] = num++;
 right--;

 for (int j = right; j ≥ left; j--)
 res[bottom][j] = num++;
 bottom--;

 for (int i = bottom; i ≥ top; i--)
 res[i][left] = num++;
 left++;
}

return res;
}

```

```

};

```

优点:

- 不需要 `loop`
- 不需要 `offset`
- 不需要特殊处理中心

----

# 八、最容易犯的 5 个错误

### ❌ 错误1

for(col=0; col<n; col++)

会 **\*\*覆盖角元素\*\***。

----

### ❌ 错误2

忘记

offset++

导致越界。

----

### ❌ 错误3

忘记

startx++ starty++

导致一直写第一圈。

----

### ❌ 错误4

奇数中心没有处理

n=1

直接挂。

----

### ❌ 错误5

第二条边写错

res[row][col]

导致列不变。

----

# 九、面试总结口诀

螺旋矩阵核心口诀：

一圈四边 左闭右开 一圈一缩 奇数补心

----

如果你愿意，我可以再给你整理一份 **《代码随想录螺旋矩阵超详细图解版》**，

把 **每一步 row/col 如何变化画出来**，基本一看就完全懂了。

很多刷 **LeetCode** 的人都是靠这个理解的

### **《代码随想录》数组：区间和**(<https://notes.kamacoder.com/questions/501952>)

这段代码的核心是在做 **前缀和 (Prefix Sum)**，用于快速计算数组某一段区间的和。

----

## # 一、代码整体作用

功能流程：

- 1 输入数组长度 `n`
- 2 输入 `n` 个数组元素
- 3 构建 **前缀和数组** `p`
- 4 不断输入区间 `[a,b]`
- 5 输出该区间的和

时间复杂度：

预处理  $O(n)$  查询  $O(1)$

----

## # 二、逐行解释代码

### 1 头文件

```
#include #include using namespace std;
```

作用:

- `iostream` → 使用 `cin`
- `vector` → 使用动态数组 `vector`

----

### 2 定义变量

```
int n, a, b; cin >> n;
```

含义:

n = 数组长度 a, b = 查询区间

例如

5

说明数组有5个元素。

----

### 3 创建数组

```
vector vec(n); vector p(n);
```

含义:

vec → 原数组 p → 前缀和数组

例如

vec: [1 2 3 4 5]

前缀和

p: [1 3 6 10 15]

----

### 4 构建前缀和

```
int presum = 0;
```

```
for (int i = 0; i < n; i++) { scanf("%d", &vec[i]); presum += vec[i]; p[i] = presum; }
```

过程:

输入数组，同时计算前缀和。

例子:

输入

1 2 3 4 5

执行过程

```
i	vec[i]	presum	p[i]
0	1	1	1
1	2	3	3
2	3	6	6
3	4	10	10
4	5	15	15
```

----

# 三、核心部分：区间求和

```
while (~scanf("%d%d", &a, &b))
```

含义：

不断读取

a b

直到 **\*\*EOF（文件结束）\*\***。

``~` 是 **\*\*按位取反\*\***，用来判断 `scanf` 是否成功读取。

如果读取成功：

scanf返回2 ~2 = -3 while(true)

如果读取失败：

scanf返回EOF(-1) ~-1 = 0 循环结束

----

#### # 四、区间和计算

```
int sum;
```

```
if (a == 0) sum = p[b]; else sum = p[b] - p[a - 1];
```

公式:

$$\text{sum}[a,b] = p[b] - p[a-1]$$

示例

数组

1 2 3 4 5

前缀和

1 3 6 10 15

求

[1,3]

计算

$$p[3] - p[0] \quad 10 - 1 = 9$$

即

$$2 + 3 + 4 = 9$$

----

# 五、输出结果

```
printf("%d\n", sum);
```

输出区间和。

----

# 六、代码运行示例

输入

5 1 2 3 4 5 0 2 1 3 2 4

数组

1 2 3 4 5

查询

0 2 -> 1+2+3=6 1 3 -> 2+3+4=9 2 4 -> 3+4+5=12

输出

----

## # 七、这段代码的3个注意事项

### ### 1 不建议混用 `cin` 和 `scanf`

代码里：

```
cin >> n scanf("%d",&vec[i])
```

虽然能运行，但**\*\*不推荐混用\*\***。

建议统一：

**\*\*全部 scanf\*\***

```
scanf("%d",&n);
```

或者

**\*\*全部 cin\*\***

```
cin >> vec[i];
```

----

### ### 2 `while(~scanf())` 写法很多人看不懂

更直观写法：

```
while (scanf("%d%d",&a,&b) != EOF)
```

----

### 3 前缀和数组可以更简单

推荐写法：

```
p[0] = vec[0];
```

```
for(int i=1;i<n;i++) p[i] = p[i-1] + vec[i];
```

----

# 八、算法核心总结

这段代码用到了经典算法：

**\*\*Prefix Sum\*\***

核心理想：

用空间换时间

原本查询区间：

$O(n)$

变成

$O(1)$

### 【《代码随想录》数组：开发商购买土地】(<https://notes.kamacoder.com/questions/501953>)  
不会等二刷

### 【《代码随想录》数组：数组总结篇】(<https://notes.kamacoder.com/questions/501954>)  
#### 数组总结

数组的重点技巧

1. 二分法
2. 双指针
3. 滑动窗口
4. 循环不变量
5. 前缀和